

AI Data Pipeline - Architecture & Production Design

Objective

Build an asynchronous, source-backed pipeline that collects AI ecosystem data, normalizes it into canonical JSON, enriches entities, applies freshness rules, and prepares the system to scale from demo batches to 500K+ records.

High-Level Flow

Sources	Async Collection	Raw Data	Extraction / Normalization
Web pages Public APIs	aiohttp + asyncio concurrent tasks	JSON / HTML source provenance	Schema validation LLM-ready extraction
Entity Resolution	Enrichment	Freshness / QA	Storage / Delivery
Canonical names mapping log	GitHub metrics paper metadata	24h checks quality flags	JSON now DB / vector / graph at scale

Implemented Components

- 1. Async crawler:** concurrent HTTP requests using asyncio/aiohttp with timeout and exception handling.
- 2. Source adapters:** GitHub, Hugging Face Hub, arXiv, TechCrunch, The Verge and public job-source pages.
- 3. Canonical schema:** schemaVersion, recordType, source, content, collectedAt.
- 4. Deterministic entity resolution:** normalization plus raw-to-canonical mapping log.
- 5. Enrichment:** GitHub repository URL and stars when an explicit repository link is available.
- 6. Freshness:** article publication dates are parsed when available; records are marked FRESH_24H or STALE_OR_UNKNOWN.
- 7. LLM orchestration:** provider configuration, structured-output validation and fallback extraction path.
- 8. Responsible anti-bot strategy:** normal HTTP responses are respected; 403/CAPTCHA/Cloudflare protections are not bypassed.

Scalable Production Architecture

500K+ Record Design

The current demo writes JSON files, but the same stages can be distributed. At production scale, collection work is partitioned by source and page ranges, queued, checkpointed, retried, and written to durable storage.

Layer	Production Design	Purpose
Scheduler	Cron / workflow engine	Trigger full loads, hourly freshness checks, incremental jobs
Queue	Kafka / SQS / RabbitMQ	Decouple producers from workers; absorb bursts
Workers	Async Python workers	Concurrent fetching, parsing and API calls
Raw storage	Object storage	Immutable HTML/API payloads for replay and auditing
Structured DB	PostgreSQL	Canonical startup/product/paper/job/news entities
Vector store	pgvector / vector DB	Semantic search over normalized content
Graph store	Neo4j / graph DB	Company-product-paper-job relationships
Observability	Metrics + logs	Latency, retries, error rates, freshness and throughput

Reliability & Backpressure

Use bounded concurrency per source to avoid overwhelming external services. Add exponential backoff for transient failures, retry budgets, and a dead-letter queue for records that repeatedly fail.

Rate Limits

Respect provider quotas and source policies. Use per-domain semaphores, request spacing, pagination, and cached responses where appropriate.

413 Payload Overflow

For oversized HTML/text sent to an LLM, strip navigation/noise first, then chunk by headings/semantic sections. Extract per chunk, validate each result, and merge deterministically.

429 Rate Limit Handling

Retry after the provider's suggested delay when available, apply exponential backoff with jitter, and fail over to the next configured LLM provider when the primary is unavailable.

Idempotency

Use stable keys based on normalized URL/entity ID plus source. A successful record can be safely reprocessed without producing duplicates.

Checkpointing

Persist page cursors, last successful fetch time, and source-level checkpoints so a failed batch can resume instead of restarting from zero.

Data Quality, LLM Orchestration & Delivery

Canonical Data Contract

Every record follows a consistent envelope: schemaVersion, recordType, source, content, and collectedAt. Missing values remain null/unknown instead of being guessed.

Record	Key fields
Startup	entityName, website/GitHub metadata, source, collectedAt
Product	productName, description, pricing, category, downloads/likes/tags
Research Paper	title, authors, abstract, paper_url, github_url, github_stars, published_date
Job	company, date, is_remote, role_family, source
News	title, description, publishedDate, freshness, source

LLM Orchestration Strategy

1. Primary provider receives cleaned, bounded input.
2. Prompt requests only the canonical JSON schema.
3. JSON is parsed and schema-validated.
4. On invalid output, retry with a stricter repair prompt.
5. On 429/timeout/provider failure, use configured fallback provider.
6. On persistent failure, use deterministic extraction and mark the record for review.

Entity Resolution Strategy

Normalize case, punctuation and whitespace, then map known aliases to canonical entities. Store both rawName and canonicalName so the transformation is auditable.

Quality Gates

Source URL present; recordType valid; required fields present; timestamps parse correctly; freshness status consistent; duplicate keys removed; failed sources retained in logs.

Submission Outputs

Artifact	Purpose
README.md	Project overview, setup, pipeline and scaling explanation
architecture.pdf	Three-page architecture and production design
data/*.json	Collected demo datasets and mapping logs
src/*.py	Collection, processing, enrichment and orchestration code

Current demonstrated scale: 1,000 startup candidates, 1,000 AI/ML product-model records, 1,000 research paper records, 20 news articles (14 fresh in the last 24 hours), and 5 job sources.

The demo emphasizes source provenance, deterministic processing, graceful failure, and a clear path to distributed production scale.